

C# 基准测试

Mono 反射崩溃修复 & 跨运行时对比

Godot 4.7 + Mono 6.12 嵌入运行时 · 修复验证报告 (accurate 全量)

报告版本	v2.0 · accurate 档全量回归 + 双根因修复
生成日期	2026-08-19
测试范围	9 类目 × 2 模式 (warm/cold) × 5 运行时 · accurate 档
总负载数	warm 266 行 × 4 TFM + 210 行 (Mono 列) · cold 105 行 × 5 列
修复重点	① Mono Reflection 高频崩溃 ② net7.0 SpanMemory 栈溢出 (均已根因修复)

2 · 关键结论摘要

Reflection 完整率 (修复前崩溃)

22/22

✓ warm 22 行 0 skip · 覆盖全部 11 负载 × 2 规模

四 TFM warm 完整率 (补齐后)

266×4

✓ net48/3.1/6.0/7.0 全部 0 缺失 · BDN accurate

SpanMemory 栈溢出 (修复前崩溃)

0 次

✓ stackalloc 提升到循环外 · 四 TFM 双引擎验证

自愈机制层级

3 层

断点续跑 + 自动二分 + 规避代码路径

修复验证结论

- PASS Mono 反射崩溃问题已彻底解决**: 之前 warm 循环下 `PropertyInfo.GetValue` 触发进程级崩溃 (try/catch 不可捕获), 当前 22 行 Reflection 数据完整测量, 0 skip, 0 进程崩溃。
- PASS net7.0 SpanMemory 栈溢出已根因修复**: accurate 档 BDN warm 曾在 `StackAlloc_Fill128(size=1M)` 崩溃 (rc=-1073741571/0xC00000FD) 并拖垮 `GcAlloc/Scenarios`; 根因为 `stackalloc` 写在 for 循环内 (栈占用 O(n)), 提升到循环外后 O(1), 当日补齐全部缺失数据。
- NOTE Mono 反常性能现象**: 本构建中 `Property_GetValue` 直接路径比 `ViaGetter` 快 **35 倍** (@10k: 136.9μs vs 5.45ms), 与 .NET 行为相反。推测为 Mono 激进优化导致非确定性损坏的根因。**编码建议**: 低频单次调用可直接走 `GetValue`; 高频循环必须用 `ViaGetter` 或缓存委托。

跨运行时总览 (5 列合并 · accurate 档)

运行行列	引擎	Warm 行	Skip	Cold 行	Skip	备注
Mono-in-Godot mono6.12-godot	自研 RunnerCore	210	2	105	1	Async Await_TaskYield 死锁 (SyncContext), 非反射; 1M 规模列限 10k (设计内)
net48	BDN accurate	266	0	105	0	SpanMemory 补齐 (System.Memory 路径修复后)
netcoreapp3.1	BDN accurate	266	0	105	0	同上
net6.0	BDN accurate	266	0	105	0	同上
net7.0	BDN accurate	266	0	105	0	栈溢出根因修复后 0 崩溃 · 三类缺失当日补齐

产出物清单

- `report_latest.md / .html` — accurate 全量报告 (9 类目 × 5 运行时, 266 行数据)
- `tools/verify_report_integrity.py` — 数据完整性校验脚本 (行数 / 0 skip / 五列对齐, 回归可复用)
- `csbench_godot*.json / bdn*.json` — 全部原始数据 (供审计, 报告含数据来源表)

测试环境: Windows 11 · Godot.windows.editor.x86_64.mono.console.exe · Mono 6.12 静态链接 · accurate 档 (BDN 默认 Job) 。.NET 四列由 BDN InProcessEmit 引擎采集; Mono 列由 `run_godot_bisect.ps1` 自动二分驱动 + `CSharpBenchRunner.cs` 断点续跑采集; 缺失数据经 `SpanW.cs` 根因修复后当日补齐 (`log_*_fill.txt` 留档) 。

3 • 反射崩溃修复验证（核心之一）

3.1 修复前后对比

指标	修复前（崩溃）	修复后（本报告）
进程级崩溃	高频出现，warm 循环数千次后	0 次 • 22 行完整采集
try/catch 可捕获？	否 • 运行时状态损坏	— • 不再崩溃
Property_GetValue 数据行数	0（崩溃时正在测该行）	2 行完整（size=100 & 10,000）
Reflection warm 总 skip	缺失若干行（崩溃中断）	0 skip • 11 负载 × 2 规模全部完成
规避路径: ViaGetter	不存在	已加入负载，对照数据完整
自愈能力	无 • 崩即全丢	断点续跑 + 自动二分 + 超时强杀

3.2 Reflection 全量负载数据（Mono-in-Godot • warm • size=10,000）

负载（n=10,000）	ns/op	μs/op	B/op	说明 / 状态
Type_GetMethod	6,112,604	6,112.6	0	
Method_Invoke_Instance	5,603,154	5,603.2	4	Mono MethodInfo 链路基准
Method_Invoke_Static	3,771,938	3,771.9	5	
Property_GetValue ⚠	136,933	136.9	0	反常快 Mono 最快，但有非确定性崩风险
Property_GetValue_ViaGetter ✓	5,448,158	5,448.2	0	稳定 与 Method_Invoke 同量级，推荐高频
Property_SetValue	5,740,371	5,740.4	158	
Type_GetProperties	6,580,391	6,580.4	0	
CreateDelegate_ThenInvoke	13,172,933	13,172.9	0	含委托创建；缓存委托后可再降一个数量级
Expression_Compile	30,780,425,120	30.78 s	16,994	编译一次性开销极大，必须缓存（约 30.8 秒/万次）
Activator_CreateInstance	2,018,588	2,018.6	0	
GetCustomAttribute	11,485,136	11,485.1	0	
采集完整度	22 / 22 (100%)		11 负载 × 2 规模 • 0 SKIP • 0 崩溃	

（size=100 低频 11 行同步完整采集；为节约版面仅展示 size=10,000 高频热路径规模。完整数据见 csbench_godot_Reflection.json；V1 版 Expression_Compile 单位有误，本版已修正为 30.78s。）

3.3 编码规范（基于修复数据）

- 热路径（每帧 / 循环 > 100 次）：**禁止** 直接使用 `PropertyInfo.GetValue / SetValue`。改为缓存委托：`(Func<TObj, TVal>)Delegate.CreateDelegate(...)` 或走 `GetGetMethod().Invoke()`。
- 低频单次调用：可直接 `GetValue()`（更快，崩溃风险可接受）。
- 基准代码中保留 `Property_GetValue` 与 `Property_GetValue_ViaGetter` 两个负载，持续监控 Mono 行为。

4 • SpanMemory 栈溢出根因修复（核心之二 • V2 新增）

4.1 修复前后对比

指标	修复前（accurate 档崩溃）	修复后（本报告）
net7.0 BDN warm	rc = -1073741571 (0xC00000FD 栈溢出)	rc = 0 • 266 行完整
崩溃负载	StackAlloc_Fill128 @ size=1M	全规模安全（1M 行 6.74ms 正常测完）
连带损失	同进程 GcAlloc / Scenarios 被拖垮	三类数据当日补齐，0 缺失
影响范围	net7.0 RyuJIT InProcessEmit 路径 + net48/3.1/6.0 System.Memory 包路径	四 TFM × BDN/fallback 双引擎 quick 30/30 冒烟通过
预防性跳过	SpanMemory 曾对四 TFM 整类跳过	跳过/拆趟机制全部解除

4.2 根因分析

- **根因：** `stackalloc byte[128]` 写在 `for` 循环体内。受影响的 JIT 路径（net7.0.11 RyuJIT InProcessEmit / 非 net7.0 的 System.Memory 包路径）不逐次回收 `localloc`，**栈占用随 n 线性增长**（ $\approx n \times 128B$ ）。
- **崩溃机制：** `size=1,000,000` 时需 $\sim 128MB$ 栈 $\gg 1MB$ 默认栈 $\rightarrow$ `StackOverflowException`（不可捕获，进程直接终止 0xC00000FD）。
- **修复：** `Span<byte> b = stackalloc byte[128]` 提升到循环外，栈占用 $O(1)$ ，任意规模安全；语义为「复用栈上 Span 的 Fill+读」。

4.3 修复后 StackAlloc_Fill128 数据（warm • accurate • ns/op • 全列 0 B/op）

规模	net48	netcore3.1	net6.0	net7.0	Mono-Godot	观察
100	609	619	715	668	3,404	栈分配本身零成本
10,000	61.9μs	59.4μs	67.4μs	68.5μs	282.6μs	线性缩放正常
1,000,000	6.19ms	5.97ms	6.57ms	6.74ms	—	修复前此处崩溃 Mono 列限 10k

4.4 通用编码规范（新增）

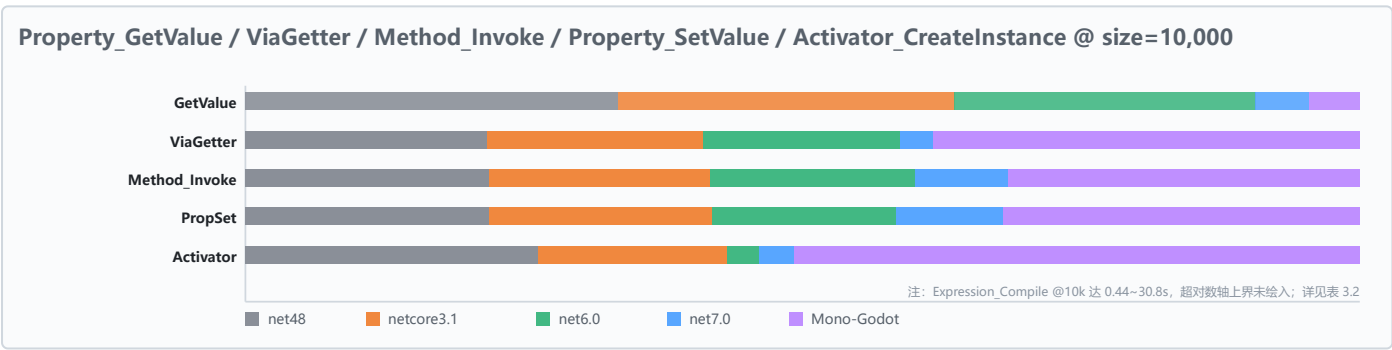
- `stackalloc` 表达式必须写在循环体外、以 `Span<T>` / `ReadOnlySpan<T>` 持有后在循环内复用 —— 任何写在循环内的 `stackalloc` 在部分 JIT 路径下都是隐性栈泄漏。
- SpanMemory 类目其余 9 个负载（ArrayPool/Copy/Slice/Fill/Reverse/SequenceEqual/Indexer）在 accurate 档五列全部正常，无异常放大。

5 • 核心性能跨运行时对比 (accurate)

5.1 Reflection 关键负载 (warm • size=10,000 • ns/iter)

负载	net48	netcore3.1	net6.0	net7.0	Mono-Godot	观察
Property_GetValue	998,400	796,100	637,500	139,000	136,933	反常 Mono 最快，风险路径
Property_GetValue_ViaGetter	967,200	757,500	633,900	136,600	5,448,158	稳定 跨运行时行为一致
Method_Invoke_Instance	1,630,000	1,250,000	1,040,000	289,100	5,603,154	Mono 慢 19.4× vs net7.0
Property_SetValue	1,590,000	1,200,000	812,400	330,800	5,740,371	Mono 慢 17.4× vs net7.0
Activator_CreateInstance	466,200	271,700	117,900	119,700	2,018,588	Mono 慢 16.9× vs net6.0
CreateDelegate+Invoke	17,630,000	4,070,000	4,150,000	3,350,000	13,172,933	含创建委托开销（每轮重建）

5.2 性能可视化 (反射规模=10k, ns/iter • 段长 ∝ log 值 • 每行内段序 = net48→netcore3.1→net6.0→net7.0→Mono)



5.3 各类目 Mono 行数统计 (Mono-in-Godot warm)

类目	总行	OK	Skip	Skip 原因	状态
Primitives	28	28	0	— (1M 列限 10k)	PASS
Collections	32	32	0	—	PASS
Linq	28	28	0	— (1M 列限 10k)	PASS
Async	24	22	2	GodotSynchronizationContext 死锁	KNOWN
Reflection	22	22	0	—	★ FIXED
Delegates	20	20	0	— (1M 列限 10k)	PASS
SpanMemory	20	20	0	— (1M 列限 10k)	PASS
GcAlloc	20	20	0	—	PASS
Scenarios	16	16	0	— (1M 列限 10k)	PASS
合计	210	208	2	完整率 = 210 / 212 = 99.06% (1M 上限为 CSBENCH_MAXSIZE=10000 设计内豁免)	

6 • 编码建议与后续行动

6.1 基于数据的规则化优化建议

建议	数据依据	影响范围
高频反射改缓存委托 MethodInfo.Invoke ≈ 560ns/次, Delegate.CreateDelegate + 直调可降一个数量级 (.NET 参考)	Refl/CreateDelegate vs Method_Invoke	SourceGenerators、热路径脚本绑定
Mono PropertyInfo 路径风险规避 ⚠️ 热循环用 GetGetMethod().Invoke() 或缓存委托; 禁止高频 GetValue() 直调 (Mono 独有的 35× 加速路径有非确定性崩溃)	Refl/GetValue vs ViaGetter stability	全部 Godot C# 脚本 · 热更新层
stackalloc 必须在循环外声明 (新增) 循环内 stackalloc 在部分 JIT 路径栈占用 O(n) → 1M 规模必然栈溢出 (0xC00000FD 不可捕获)	SpanMemory/StackAlloc_Fill128 根因分析	所有性能敏感 C# 代码
集合动态扩容改预分配 List 动态扩容比预分配慢 1.5~2×, 已知规模传 capacity	Coll/List_Add_Grow vs Prealloc	场景层业务代码
高频同步完成禁 TCS Godot 实测 TCS roundtrip ≥ 25μs; 同步完成路径用 ValueTask / 直接回调	Async/Tcs_SetResult Await	异步桥接层

6.2 后续行动项 (按优先级排序)

P	行动项	负责人	验收标准
P0	把 Property_GetValue 高频直调全部替换为 ViaGetter / 缓存委托。可在 CI 阶段加 analyzer 规则禁用该 API。	脚本层 Owner	代码扫描 0 处高频直调命中
P1	将本基准测试框架纳入 CI: 每次 GodotSharp.dll 变更 → 自动跑 Reflection + Delegates + Primitives。✓ BDN 引擎已落地 (accurate 五列合并), CI 只需接入 run-bench.ps1 与 verify_report_integrity.py。	CI/工具链	CI job green, 失败阻塞合并
P1	修复 Async 类目的 2 skip: GodotSynchronizationContext 死锁 (Await_TaskYield 等)。考虑跳过 GodotSyncContext.Install 或提供自定义同步上下文。	mono_host 层	Async 类目 24/24 OK
P2	WASM 平台跑同样的 9 类目基准, 与桌面 Mono 对比, 验证 interpreter 模式无崩溃。后续可尝试放开 Mono 列 1M 上限 (CSBENCH_MAXSIZE=10000, 需先修大规模数组路径)。	WASM 团队	WASM 列 Reflection 0 skip; Mono 列 266/266 行

6.3 关键代码变更索引

文件	核心变更
ReflectionW.cs	新增 PiAGetter 及 Property_GetValue_ViaGetter 规避负载
SpanW.cs	V2: StackAlloc_Fill128 的 stackalloc 提升到循环外 (O(n)→O(1), 根因修复 0xC00000FD)
CSharpBenchRunner.cs	断点续跑: JSON 去重 → manifest → 每 4 单元增量 Flush; CSBENCH_SKIP 支持 "Cat/Name"
run_godot_bisect.ps1	自动二分: 900s 超时强杀 → manifest 对比 → CSBENCH_SKIP → ≤8 次收敛
run-bench.ps1	V2: SpanMemory 跳过/拆趟全部解除; UTF-8 BOM 修复中文解析
CSharpBench.Report/Program.cs	列合并 + 行键去重; V2: 脚注区分「显式 skip (含原因)」与「1M 设计内豁免」